

Week 11 Skill

To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor 4 Components

```
divya@Divyasree:~$ cd ~/job_listing
divya@Divyasree:~/job_listing$ ls
foreground      process_groups  signals.c
foreground.c    process_groups.c signals.c.save
interrupt_job   resume_jobs    suspend_job
interrupt_job.c resume_jobs.c  suspend_job.c
jobs           signal_handlers suspend_job.c.save
jobs.c         signal_handlers.c
jobs.c.save    signals
divya@Divyasree:~/job_listing$ nano integrated_shell.c
divya@Divyasree:~/job_listing$ gcc integrated_shell.c -o integrated_shell
divya@Divyasree:~/job_listing$ ./integrated_shell
=====
      INTEGRATED JOB CONTROL SYSTEM
=====

===== MAIN MENU =====
1. Start Foreground Job
2. List Jobs
3. Test Job Recovery
4. Exit
Enter choice: 1

[Shell] Job created.
[Shell] Job ID: 1
[Shell] PID: 5032
[Shell] Job State: Running

Press Ctrl+C to terminate
or Ctrl+Z to stop the job.

[Child] Job started.
[Child] PID: 5032
```

```
[Child] Job started.
[Child] PID: 5032
[Child] Working... 1/15
[Child] Working... 2/15
[Child] Working... 3/15
[Child] Working... 4/15
[Child] Working... 5/15
[Child] Working... 6/15
[Child] Working... 7/15
[Child] Working... 8/15
[Child] Working... 9/15
[Child] Working... 10/15
[Child] Working... 11/15
[Child] Working... 12/15
[Child] Working... 13/15
[Child] Working... 14/15
[Child] Working... 15/15
[Child] Job completed.
[Job Table] Job 1 -> Completed

===== MAIN MENU =====
1. Start Foreground Job
2. List Jobs
3. Test Job Recovery
4. Exit
Enter choice: ^C
[Signal] SIGINT received by shell.
^C
[Signal] SIGINT received by shell.
|
```

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_JOBS 10

typedef enum {
    RUNNING,
    STOPPED,
    COMPLETED,
    TERMINATED
} JobState;

typedef struct {
    int job_id;
    pid_t pid;
    char command[50];
    JobState state;
} Job;

Job job_table[MAX_JOBS];
int job_count = 0;

pid_t foreground_pid = -1;

/* -----
   Component 1: Job Management
   ----- */

const char *state_name(JobState state)
{
    switch (state) {
        case RUNNING:
            return "Running";
        case STOPPED:
            return "Stopped";
        case COMPLETED:
            return "Completed";
```

```

        case TERMINATED:
            return "Terminated";
        default:
            return "Unknown";
    }
}

int add_job(pid_t pid, const char *command)
{
    if (job_count >= MAX_JOBS) {
        printf("[Error] Job table is full.\n");
        return -1;
    }

    job_table[job_count].job_id = job_count + 1;
    job_table[job_count].pid = pid;

    strncpy(job_table[job_count].command,
            command,
            sizeof(job_table[job_count].command) - 1);

    job_table[job_count].command[
        sizeof(job_table[job_count].command) - 1
    ] = '\0';

    job_table[job_count].state = RUNNING;

    job_count++;

    return job_count - 1;
}

void update_job_state(pid_t pid, JobState state)
{
    for (int i = 0; i < job_count; i++) {
        if (job_table[i].pid == pid) {
            job_table[i].state = state;

            printf("[Job Table] Job %d -> %s\n",
                    job_table[i].job_id,
                    state_name(state));

            return;
        }
    }
}

```

```

    }
}

printf("[Error] Job with PID %d not found.\n", pid);
}

void list_jobs()
{
    printf("\n===== JOB TABLE =====\n");

    if (job_count == 0) {
        printf("No jobs available.\n");
        return;
    }

    printf("%-8s %-10s %-20s %-12s\n",
        "Job ID",
        "PID",
        "Command",
        "State");

    printf("-----\n");

    for (int i = 0; i < job_count; i++) {
        printf("%-8d %-10d %-20s %-12s\n",
            job_table[i].job_id,
            job_table[i].pid,
            job_table[i].command,
            state_name(job_table[i].state));
    }
}

/* -----
Component 2: Signal Handling
----- */

void handle_sigint(int sig)
{
    printf("\n[Signal] SIGINT received by shell.\n");

    if (foreground_pid > 0) {

        printf("[Signal] Forwarding SIGINT to PID %d.\n",

```

```

        foreground_pid);

    kill(foreground_pid, SIGINT);
}
}

void handle_sigstsp(int sig)
{
    printf("\n[Signal] SIGTSTP received by shell.\n");

    if (foreground_pid > 0) {

        printf("[Signal] Forwarding SIGTSTP to PID %d.\n",
            foreground_pid);

        kill(foreground_pid, SIGTSTP);
    }
}

/* -----
Component 3: Start Job
----- */

pid_t start_job()
{
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return -1;
    }

    if (pid == 0) {

        /* Child process */

        signal(SIGINT, SIG_DFL);
        signal(SIGTSTP, SIG_DFL);

        printf("\n[Child] Job started.\n");
        printf("[Child] PID: %d\n", getpid());

        for (int i = 1; i <= 15; i++) {

```

```

        printf("[Child] Working... %d/15\n", i);

        sleep(1);
    }

    printf("[Child] Job completed.\n");

    exit(0);
}

return pid;
}

/* -----
Component 4: Execution / Recovery
----- */

void execute_foreground_job()
{
    pid_t pid;
    int status;

    pid = start_job();

    if (pid < 0)
        return;

    int index = add_job(pid, "demo-job");

    if (index < 0) {
        kill(pid, SIGTERM);
        return;
    }

    foreground_pid = pid;

    printf("\n[Shell] Job created.\n");
    printf("[Shell] Job ID: %d\n",
        job_table[index].job_id);

    printf("[Shell] PID: %d\n", pid);

```

```

printf("[Shell] Job State: Running\n");

printf("\nPress Ctrl+C to terminate\n");
printf("or Ctrl+Z to stop the job.\n");

waitpid(pid, &status, WUNTRACED);

if (WIFSTOPPED(status)) {

    update_job_state(pid, STOPPED);

    printf("\n[Shell] Job is stopped.\n");

    printf("Resume job now? (y/n): ");

    char choice;
    scanf(" %c", &choice);

    if (choice == 'y' || choice == 'Y') {

        printf("[Shell] Sending SIGCONT...\n");

        kill(pid, SIGCONT);

        update_job_state(pid, RUNNING);

        waitpid(pid, &status, 0);
    }
}

if (WIFEXITED(status)) {

    update_job_state(pid, COMPLETED);
}
else if (WIFSIGNALED(status)) {

    update_job_state(pid, TERMINATED);
}

foreground_pid = -1;
}

/* -----

```

Main Integration

```
----- */

int main()
{
    int choice;

    printf("=====\n");
    printf("  INTEGRATED JOB CONTROL SYSTEM\n");
    printf("=====\n");

    /* Register shell signal handlers */

    signal(SIGINT, handle_sigint);
    signal(SIGTSTP, handle_sigtstp);

    while (1) {

        printf("\n===== MAIN MENU =====\n");
        printf("1. Start Foreground Job\n");
        printf("2. List Jobs\n");
        printf("3. Test Job Recovery\n");
        printf("4. Exit\n");
        printf("Enter choice: ");

        if (scanf("%d", &choice) != 1) {

            printf("[Error] Invalid input.\n");

            while (getchar() != '\n');

            continue;
        }

        switch (choice) {

            case 1:
                execute_foreground_job();
                break;

            case 2:
                list_jobs();
                break;
```


case 3:

```
printf("\n[Recovery] Checking job states...\n");
```

```
list_jobs();
```

```
break;
```

case 4:

```
printf("\n[Shell] Cleaning up...\n");
```

```
for (int i = 0; i < job_count; i++) {
```

```
    if (job_table[i].state == RUNNING ||  
        job_table[i].state == STOPPED) {
```

```
        kill(job_table[i].pid, SIGTERM);
```

```
    }
```

```
}
```

```
printf("[Shell] Exiting safely.\n");
```

```
return 0;
```

default:

```
printf("[Error] Invalid menu choice.\n");
```

```
}
```

```
}
```

```
return 0;
```

```
}
```

TO Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios

```

divya@Divyasree: ~/error_har
divya@Divyasree:~$ mkdir -p ~/error_handling
cd ~/error_handling
divya@Divyasree:~/error_handling$ pwd
/home/divya/error_handling
divya@Divyasree:~/error_handling$ nano error_handler.c
divya@Divyasree:~/error_handling$ gcc -Wall -Wextra error_handler.c -o error_handler
divya@Divyasree:~/error_handling$ ls
error_handler  error_handler.c
divya@Divyasree:~/error_handling$ ./error_handler
=====
ERROR HANDLING MANAGEMENT SYSTEM
=====

===== ERROR HANDLING SYSTEM =====
1. Perform Division
2. Test Invalid Input
3. Test File Error
4. View Error Log
5. Exit
=====
Enter your choice: 1

Enter first number: 10
Enter second number: 0
Error: Runtime error: division by zero is not allowed.

===== ERROR HANDLING SYSTEM =====
1. Perform Division
2. Test Invalid Input
3. Test File Error
4. View Error Log
5. Exit
=====

Enter your choice: 1

Enter first number: 20
Enter second number: 5
Result = 4.00

===== ERROR HANDLING SYSTEM =====
1. Perform Division
2. Test Invalid Input
3. Test File Error
4. View Error Log
5. Exit
=====

Enter your choice: 2

Enter an integer: hello
Error: Invalid syntax: expected an integer.

===== ERROR HANDLING SYSTEM =====
1. Perform Division
2. Test Invalid Input
3. Test File Error
4. View Error Log
5. Exit
=====

Enter your choice: 3
Error: File error: unable to open missing_file.txt: No such file or directory
Recovering gracefully... continuing program.

===== ERROR HANDLING SYSTEM =====
1. Perform Division
2. Test Invalid Input
3. Test File Error

```

```

2. Test Invalid Input
3. Test File Error
4. View Error Log
5. Exit
=====
Enter your choice: abc
Error: Invalid syntax: menu choice must be a number.
Recovering gracefully...

===== ERROR HANDLING SYSTEM =====
1. Perform Division
2. Test Invalid Input
3. Test File Error
4. View Error Log
5. Exit
=====
Enter your choice: 5

Program terminated successfully.
All detected failures were logged in error.log.
divya@Divyasree:~/error_handling$ cat error.log
[Mon Sep 28 04:54:55 2026
] Runtime error: division by zero is not allowed.
[Mon Sep 28 04:55:16 2026
] Invalid syntax: expected an integer.
[Mon Sep 28 04:55:26 2026
] File error: unable to open missing_file.txt: No such file or directory
[Mon Sep 28 04:55:36 2026
] Invalid menu choice.
[Mon Sep 28 04:55:39 2026
] Invalid syntax: menu choice must be a number.
divya@Divyasree:~/error_handling$ ls -l error.log
-rw-r--r-- 1 divya divya 363 Sep 28 04:55 error.log

```

```

5. Exit
=====
Enter your choice: 4

===== ERROR LOG =====
[Mon Sep 28 04:54:55 2026
] Runtime error: division by zero is not allowed.
[Mon Sep 28 04:55:16 2026
] Invalid syntax: expected an integer.
[Mon Sep 28 04:55:26 2026
] File error: unable to open missing_file.txt: No such file or directory
[Mon Sep 28 04:55:36 2026
] Invalid menu choice.
[Mon Sep 28 04:55:39 2026
] Invalid syntax: menu choice must be a number.
=====

===== ERROR HANDLING SYSTEM =====
1. Perform Division
2. Test Invalid Input
3. Test File Error
4. View Error Log
5. Exit
=====
Enter your choice: ^C
divya@Divyasree:~/error_handling$ ls -l
total 32
-rw-r--r-- 1 divya divya 363 Sep 28 04:55 error.log
-rwxr-xr-x 1 divya divya 16832 Sep 28 04:54 error_handler
-rw-r--r-- 1 divya divya 4133 Sep 28 04:54 error_handler.c
divya@Divyasree:~/error_handling$ |

```

Code:

```
error_handler.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <time.h>

#define MAX_INPUT 100

/* Log error messages into error.log */
void log_error(const char *message)
{
    FILE *log = fopen("error.log", "a");
    if (log == NULL)
    {
        printf("ERROR: Unable to open error.log\n");
        return;
    }
    time_t now = time(NULL);
    #printf(log, "[%s] %s\n", ctime(&now), message);
    #fclose(log);
}

/* Display and log an error */
void report_error(const char *message)
{
    printf("Error: %s\n", message);
    log_error(message);
}

/* Perform division with runtime error checking */
int perform_division()
{
    double a, b;

    printf("\nEnter first number: ");
    if (scanf("%lf", &a) != 1)
    {
        report_error("Invalid syntax: first input is not a number.");
        while (getchar() != '\n')
            ;
        return;
    }

    printf("Enter second number: ");
    if (scanf("%lf", &b) != 1)
    {
        report_error("Invalid syntax: second input is not a number.");
        while (getchar() != '\n')
            ;
        return;
    }

    if (b == 0)
    {
        report_error("Runtime error: division by zero is not allowed.");
        return;
    }

    printf("Result = %.2lf\n", a / b);
}

/* Test file error */
void test_file_error()
{
    FILE *file = fopen("missing_file.txt", "r");
    if (file == NULL)
    {
        char message[MAX_INPUT];
        sprintf(message, "%s", "File error: unable to open missing_file.txt: %s",
            strerror(errno));
        report_error(message);
        printf("Recovering gracefully... continuing program.\n");
        return;
    }
    #fclose(file);
}

/* Test invalid input */
void test_invalid_input()
{
    char input[MAX_INPUT];
    int number;

    printf("\nEnter an integer: ");
    scanf("%d", &number);
}
```

```

GNU nano 7.2 error_handler.c
    }
    printf("Valid integer entered: %d\n", number);
}

/* Display menu */
void display_menu()
{
    printf("\n===== ERROR HANDLING SYSTEM =====\n");
    printf("1. Perform Division\n");
    printf("2. Test Invalid Input\n");
    printf("3. Test File Error\n");
    printf("4. View Error Log\n");
    printf("5. Exit\n");
    printf("===== \n");
    printf("Enter your choice: ");
}

/* Display error log */
void view_log()
{
    FILE *log = fopen("error.log", "r");
    if (log == NULL)
    {
        printf("\nNo error log exists yet.\n");
        return;
    }
    char line[300];
    printf("\n===== ERROR LOG =====\n");
    while (fgets(line, sizeof(line), log) != NULL)
    {
        printf("%s", line);
    }
    printf("===== \n");
    fclose(log);
}

int main()
{
    int choice;
    printf("===== \n");
    printf("  ERROR HANDLING MANAGEMENT SYSTEM  \n");
    printf("===== \n");
    while (1)
    {
        display_menu();
        if (scanf("%d", &choice) != 1)
        {
            report_error("Invalid syntax: menu choice must be a number.");
            printf("Recovering gracefully...\n");
            while (getchar() != '\n')
                ;
            continue;
        }
        switch (choice)
        {
            case 1:
                perform_division();
                break;
            case 2:
                test_invalid_input();
                break;
            case 3:
                test_file_error();
                break;
            case 4:
                view_log();
                break;
            case 5:
                printf("\nProgram terminated successfully.\n");
                printf("All detected failures were logged in error.log.\n");
                return 0;
            default:
                report_error("Invalid menu choice.");
                printf("Please select a valid option from 1 to 5.\n");
        }
    }
    return 0;
}

```

To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings

```

divya@Divyasree:~$ mkdir -p ~/valgrind_memory
cd ~/valgrind_memory
divya@Divyasree:~/valgrind_memory$ pwd
/home/divya/valgrind_memory
divya@Divyasree:~/valgrind_memory$ nano memory_check.c
divya@Divyasree:~/valgrind_memory$ gcc -g -Wall -Wextra memory_check.c -o memory_check
divya@Divyasree:~/valgrind_memory$ ls
memory_check  memory_check.c
divya@Divyasree:~/valgrind_memory$ ./memory_check
=====
VALGRIND MEMORY MANAGEMENT TEST
=====

--- Memory Allocation Test ---
String: Valgrind Memory Test
Numbers: 10 20 30 40 50
Memory released successfully.

--- Dynamic Array Test ---
Array values: 0 1 4 9 16 25 36 49 64 81
Dynamic array memory released.

--- Reallocation Test ---
Initial values: 1 2 3
After realloc: 1 2 3 4 5 6
Reallocated memory released.

All dynamically allocated memory has been released.
Program completed successfully.
divya@Divyasree:~/valgrind_memory$ valgrind --version
valgrind-3.22.0
divya@Divyasree:~/valgrind_memory$ valgrind --leak-check=full ./memory_check
==751== Memcheck, a memory error detector
=====
VALGRIND MEMORY MANAGEMENT TEST
=====

--- Memory Allocation Test ---
String: Valgrind Memory Test
Numbers: 10 20 30 40 50
Memory released successfully.

--- Dynamic Array Test ---
Array values: 0 1 4 9 16 25 36 49 64 81
Dynamic array memory released.

--- Reallocation Test ---
Initial values: 1 2 3
After realloc: 1 2 3 4 5 6
Reallocated memory released.

All dynamically allocated memory has been released.
Program completed successfully.
==751==
==751== HEAP SUMMARY:
==751==   in use at exit: 0 bytes in 0 blocks
==751==   total heap usage: 6 allocs, 6 frees, 1,170 bytes allocated
==751==
==751== All heap blocks were freed -- no leaks are possible
==751==
==751== For lists of detected and suppressed errors, rerun with: -s
==751== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
divya@Divyasree:~/valgrind_memory$ valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./memory_check
==756== Memcheck, a memory error detector
==756== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==756== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
=====
VALGRIND MEMORY MANAGEMENT TEST
=====

--- Memory Allocation Test ---
String: Valgrind Memory Test
Numbers: 10 20 30 40 50
Memory released successfully.

--- Dynamic Array Test ---
Array values: 0 1 4 9 16 25 36 49 64 81
Dynamic array memory released.

--- Reallocation Test ---
Initial values: 1 2 3
After realloc: 1 2 3 4 5 6
Reallocated memory released.

All dynamically allocated memory has been released.
Program completed successfully.
==756==
==756== HEAP SUMMARY:
==756==   in use at exit: 0 bytes in 0 blocks
==756==   total heap usage: 6 allocs, 6 frees, 1,170 bytes allocated
==756==
==756== All heap blocks were freed -- no leaks are possible
==756==
==756== For lists of detected and suppressed errors, rerun with: -s
==756== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
divya@Divyasree:~/valgrind_memory$ valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./memory_check 2> valgrind_re
port.txt
=====
VALGRIND MEMORY MANAGEMENT TEST
=====

```

```

Initial values: 1 2 3
After realloc: 1 2 3 4 5 6
Reallocated memory released.

All dynamically allocated memory has been released.
Program completed successfully.
divya@Divyasree:~/valgrind_memory$ ls
memory_check  memory_check.c  valgrind_report.txt
divya@Divyasree:~/valgrind_memory$ cat valgrind_report.txt
==757== Memcheck, a memory error detector
==757== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==757== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==757== Command: ./memory_check
==757==
==757==
==757== HEAP SUMMARY:
==757==   in use at exit: 0 bytes in 0 blocks
==757==   total heap usage: 6 allocs, 6 frees, 1,170 bytes allocated
==757==
==757== All heap blocks were freed -- no leaks are possible
==757==
==757== For lists of detected and suppressed errors, rerun with: -s
==757== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
divya@Divyasree:~/valgrind_memory$ q
Command 'q' not found, but can be installed with:
sudo snap install q # version 1.6.3-1, or
sudo apt install python3-q-text-as-data # version 3.1.6-3
See 'snap info q' for additional versions.
divya@Divyasree:~/valgrind_memory$ nano findings.txt
divya@Divyasree:~/valgrind_memory$ ls -l
total 32
-rw-r--r-- 1 divya divya 1106 Sep 28 05:03 findings.txt
-rwxr-xr-x 1 divya divya 18800 Sep 28 05:01 memory_check

divya@Divyasree:~/valgrind_memory$ ls
memory_check  memory_check.c  valgrind_report.txt
divya@Divyasree:~/valgrind_memory$ cat valgrind_report.txt
==757== Memcheck, a memory error detector
==757== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==757== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==757== Command: ./memory_check
==757==
==757==
==757== HEAP SUMMARY:
==757==   in use at exit: 0 bytes in 0 blocks
==757==   total heap usage: 6 allocs, 6 frees, 1,170 bytes allocated
==757==
==757== All heap blocks were freed -- no leaks are possible
==757==
==757== For lists of detected and suppressed errors, rerun with: -s
==757== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
divya@Divyasree:~/valgrind_memory$ q
Command 'q' not found, but can be installed with:
sudo snap install q # version 1.6.3-1, or
sudo apt install python3-q-text-as-data # version 3.1.6-3
See 'snap info q' for additional versions.
divya@Divyasree:~/valgrind_memory$ nano findings.txt
divya@Divyasree:~/valgrind_memory$ ls -l
total 32
-rw-r--r-- 1 divya divya 1106 Sep 28 05:03 findings.txt
-rwxr-xr-x 1 divya divya 18800 Sep 28 05:01 memory_check
-rw-r--r-- 1 divya divya 2895 Sep 28 05:01 memory_check.c
-rw-r--r-- 1 divya divya 595 Sep 28 05:02 valgrind_report.txt
divya@Divyasree:~/valgrind_memory$ ./memory_check
=====
VALGRIND MEMORY MANAGEMENT TEST
=====

```

```

--- Memory Allocation Test ---
String: Valgrind Memory Test
Numbers: 10 20 30 40 50
Memory released successfully.

--- Dynamic Array Test ---
Array values: 0 1 4 9 16 25 36 49 64 81
Dynamic array memory released.

--- Reallocation Test ---
Initial values: 1 2 3
After realloc: 1 2 3 4 5 6
Reallocated memory released.

All dynamically allocated memory has been released.
Program completed successfully.
==786==
==786== HEAP SUMMARY:
==786==   in use at exit: 0 bytes in 0 blocks
==786==   total heap usage: 6 allocs, 6 frees, 1,170 bytes allocated
==786==
==786== All heap blocks were freed -- no leaks are possible
==786==
==786== For lists of detected and suppressed errors, rerun with: -s
==786== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
divya@Divyasree:~/valgrind_memory$ |

```

Code:

```

GNU nano 7.2 memory_chi
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define SIZE 50

void allocate_memory()
{
    char *name;
    int *numbers;

    printf("\n--- Memory Allocation Test ---\n");

    name = (char *)malloc(SIZE * sizeof(char));
    numbers = (int *)malloc(5 * sizeof(int));

    if (name == NULL || numbers == NULL)
    {
        printf("Memory allocation failed.\n");
        free(name);
        free(numbers);
        return;
    }

    strcpy(name, "Valgrind Memory Test");

    for (int i = 0; i < 5; i++)
    {
        numbers[i] = (i + 1) * 10;
    }

    printf("String: %s\n", name);

    printf("Numbers: ");
    for (int i = 0; i < 5; i++)
    {
        printf("%d ", numbers[i]);
    }

    printf("\n");

    /*
     * Release allocated memory
     */
    free(name);
    free(numbers);

    /*
     * Set pointers to NULL after freeing
     */
    name = NULL;
    numbers = NULL;

    printf("Memory released successfully.\n");
}

void allocate_array()
{
    int *array;

```



```

GNU nano 7.2 mem
{
    printf("%d ", array[i]);
}

printf("\n");

/*
 * Release calloc memory
 */
free(array);
array = NULL;

printf("Dynamic array memory released.\n");
}

void allocate_and_resize()
{
    int *data;

    printf("\n--- Reallocation Test ---\n");

    data = (int *)malloc(3 * sizeof(int));

    if (data == NULL)
    {
        printf("Initial allocation failed.\n");
        return;
    }

    for (int i = 0; i < 3; i++)
    {
        data[i] = i + 1;
    }

    printf("Initial values: ");

    for (int i = 0; i < 3; i++)
    {
        printf("%d ", data[i]);
    }

    printf("\n");

    int *temp = realloc(data, 6 * sizeof(int));

    if (temp == NULL)
    {
        printf("Reallocation failed.\n");

        /*
         * Original memory is still valid,
         * so free it.
         */
        free(data);
        return;
    }

    data = temp;

    for (int i = 3; i < 6; i++)
    {
        data[i] = i + 1;
    }

    int *temp = realloc(data, 6 * sizeof(int));

    if (temp == NULL)
    {
        printf("Reallocation failed.\n");

        /*
         * Original memory is still valid,
         * so free it.
         */
        free(data);
        return;
    }

    data = temp;

    for (int i = 3; i < 6; i++)
    {
        data[i] = i + 1;
    }

    printf("After realloc: ");

    for (int i = 0; i < 6; i++)
    {
        printf("%d ", data[i]);
    }

    printf("\n");

    free(data);
    data = NULL;

    printf("Reallocated memory released.\n");
}

int main()
{
    printf("===== \n");
    printf("      VALGRIND MEMORY MANAGEMENT TEST      \n");
    printf("===== \n");

    allocate_memory();
    allocate_array();
    allocate_and_resize();

    printf("\nAll dynamically allocated memory has been released.\n");
    printf("Program completed successfully.\n");

    return 0;
}

```

To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports

```
divya@Divyasree:~$ mkdir -p ~/automated_testing
cd ~/automated_testing
divya@Divyasree:~/automated_testing$ pwd
/home/divya/automated_testing
divya@Divyasree:~/automated_testing$ nano calculator.c
divya@Divyasree:~/automated_testing$ gcc -Wall -Wextra calculator.c -o calculator
divya@Divyasree:~/automated_testing$ ls
calculator  calculator.c
divya@Divyasree:~/automated_testing$ ./calculator
Simple Calculator
1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter choice: 1
Enter two numbers: 10
20
Result: 30.00
divya@Divyasree:~/automated_testing$ ./calculator
Simple Calculator
1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter choice: 4
Enter two numbers: 10 2
Result: 5.00
divya@Divyasree:~/automated_testing$ nano test_script.sh
divya@Divyasree:~/automated_testing$ chmod +x test_script.sh
divya@Divyasree:~/automated_testing$ ls -l test_script.sh
-rwxr-xr-x 1 divya divya 1906 Sep 28 05:11 test_script.sh
divya@Divyasree:~/automated_testing$ ./test_script.sh
=====
Actual:   Simple Calculator
1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter choice: Enter two numbers: Result: 30.00
STATUS: PASS

Running: Subtraction Test
Expected: Result: 30.00
Actual:   Simple Calculator
1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter choice: Enter two numbers: Result: 30.00
STATUS: PASS

Running: Multiplication Test
Expected: Result: 30.00
Actual:   Simple Calculator
1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter choice: Enter two numbers: Result: 30.00
STATUS: PASS

Running: Division Test
Expected: Result: 4.00
Actual:   Simple Calculator
1. Addition
2. Subtraction
```

```
Running: Division By Zero Test
Expected: ERROR: Division by zero
Actual:   Simple Calculator
1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter choice: Enter two numbers: ERROR: Division by zero
STATUS: PASS
```

```
Running: Invalid Operation Test
Expected: ERROR: Invalid operation
Actual:   Simple Calculator
1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter choice: Enter two numbers: ERROR: Invalid operation
STATUS: PASS
```

```
=====
                        TEST SUMMARY
=====
Total Tests : 6
Passed      : 6
Failed      : 0
=====
```

```
Test report generated: test_report.txt
divya@Divyasree:~/automated_testing$ cat test_report.txt
Automated Test Report
=====
```

```
Running: Division Test
Expected: Result: 4.00
Actual:   Simple Calculator
1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter choice: Enter two numbers: Result: 4.00
STATUS: PASS
```

```
Running: Division By Zero Test
Expected: ERROR: Division by zero
Actual:   Simple Calculator
1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter choice: Enter two numbers: ERROR: Division by zero
STATUS: PASS
```

```
Running: Invalid Operation Test
Expected: ERROR: Invalid operation
Actual:   Simple Calculator
1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter choice: Enter two numbers: ERROR: Invalid operation
```

```
Test report generated: test_report.txt
divya@Divyasree:~/automated_testing$ cat test_report.txt
Automated Test Report
=====

Addition Test : PASS
Subtraction Test : PASS
Multiplication Test : PASS
Division Test : PASS
Division By Zero Test : PASS
Invalid Operation Test : PASS

=====
TOTAL TESTS : 6
PASSED      : 6
FAILED      : 0
=====
divya@Divyasree:~/automated_testing$ |
```

Code :

```
#include <stdio.h>

int main()
{
    int choice;
    double a, b, result;

    printf("Simple Calculator\n");
    printf("1. Addition\n");
    printf("2. Subtraction\n");
    printf("3. Multiplication\n");
    printf("4. Division\n");
    printf("Enter choice: ");

    if (scanf("%d", &choice) != 1)
    {
        printf("ERROR: Invalid choice\n");
        return 1;
    }

    printf("Enter two numbers: ");

    if (scanf("%lf %lf", &a, &b) != 2)
    {
        printf("ERROR: Invalid numbers\n");
        return 1;
    }

    switch (choice)
    {
        case 1:
            result = a + b;
            printf("Result: %.2lf\n", result);
            break;

        case 2:
```

```
GNU nano 7.2 calculator.c
switch (choice)
{
    case 1:
        result = a + b;
        printf("Result: %.2lf\n", result);
        break;

    case 2:
        result = a - b;
        printf("Result: %.2lf\n", result);
        break;

    case 3:
        result = a * b;
        printf("Result: %.2lf\n", result);
        break;

    case 4:
        if (b == 0)
        {
            printf("ERROR: Division by zero\n");
            return 1;
        }

        result = a / b;
        printf("Result: %.2lf\n", result);
        break;

    default:
        printf("ERROR: Invalid operation\n");
        return 1;
}

return 0;
}
```

Txt:

```

GNU nano 7.2 test_script
#!/bin/bash

PROGRAM="./calculator"
REPORT="test_report.txt"

PASS=0
FAIL=0
TOTAL=0

echo "=====
echo "      AUTOMATED TESTING SYSTEM"
echo "=====

echo "Automated Test Report" > "$REPORT"
echo "===== >> "$REPORT"
echo "" >> "$REPORT"

run_test()
{
    TEST_NAME="$1"
    INPUT="$2"
    EXPECTED="$3"

    TOTAL=$((TOTAL + 1))

    echo ""
    echo "Running: $TEST_NAME"

    ACTUAL=$(printf "%b" "$INPUT" | $PROGRAM)

    echo "Expected: $EXPECTED"
    echo "Actual: $ACTUAL"

    if echo "$ACTUAL" | grep -qF "$EXPECTED"
    then
        echo "STATUS: PASS"
        echo "$TEST_NAME : PASS" >> "$REPORT"
        PASS=$((PASS + 1))
    else
        echo "STATUS: FAIL"
        echo "$TEST_NAME : FAIL" >> "$REPORT"
        FAIL=$((FAIL + 1))
    fi
}

# Test Case 1
run_test \
"Addition Test" \
"1\n10 20\n" \
"Result: 30.00"

# Test Case 2
run_test \
"Subtraction Test" \
"2\n50 20\n" \
"Result: 30.00"

# Test Case 3
run_test \
"Multiplication Test" \
"3\n5 6\n" \
"Result: 30.00"

# Test Case 4
run_test \
"Division Test" \
"4\n20 5\n" \
"Result: 4.00"

# Test Case 5
run_test \
"Division By Zero Test" \
"4\n20 0\n" \
"ERROR: Division by zero"

# Test Case 6
run_test \
"Invalid Operation Test" \
"9\n10 20\n" \
"ERROR: Invalid operation"

echo "" >> "$REPORT"
echo "===== >> "$REPORT"
echo "TOTAL TESTS : $TOTAL" >> "$REPORT"
echo "PASSED      : $PASS" >> "$REPORT"
echo "FAILED      : $FAIL" >> "$REPORT"
echo "===== >> "$REPORT"

echo ""
echo "=====
echo "      TEST SUMMARY"
echo "=====
echo "Total Tests : $TOTAL"
echo "Passed      : $PASS"
echo "Failed      : $FAIL"
echo "=====

echo ""
echo "Test report generated: $REPORT"

```